

AI Gym & Fitness Assistant

Internship Project Report

Developed as part of the Industry Internship at
Unlox Academy and Trivion Technology

Prepared by: Navyashree K N
Role: Project Intern

1. Project Overview

The AI Gym & Fitness Assistant is a unified, AI-powered fitness platform developed during the internship at Unlox Academy and Trivion Technology. Instead of treating workout tracking, diet planning, motivation and equipment monitoring as separate tools, it brings them into one system that acts as a personal trainer, dietician, motivator and data-driven fitness manager.

It is built around seven AI modules: computer-vision workout detection, LLM-driven diet coaching, IoT-based smart gym monitoring over MQTT, behavioural risk prediction, a conversational fitness companion, automated performance scoring, and a recommendation engine. A FastAPI backend serves them, and the user reaches them through a deployed Streamlit dashboard (and a Next.js frontend for the core modules). Sessions, risk checks, chat moods and meals are stored in a SQLite database, which powers a weekly report, an Admin Analytics view and history-aware recommendations.

2. Objectives

- Design and implement a modular AI ecosystem covering detection, diet, behaviour, IoT-connected hardware and conversational support.
- Show real integration between the computer-vision module and the backend, with no manual data entry.
- Store usage data so the system can report progress over time and adapt its recommendations.
- Deploy the system publicly so it can be demonstrated from any device.
- Test each module independently and as a whole, and document the results honestly.

3. System Architecture

The system has a frontend layer (Streamlit and Next.js), a FastAPI backend hosted on Render, seven AI modules, a SQLite storage layer, and external services (Google Gemini, the MediaPipe pose model and an MQTT broker). The Streamlit dashboard is hosted on Streamlit Community Cloud, and a Docker Compose setup is included.

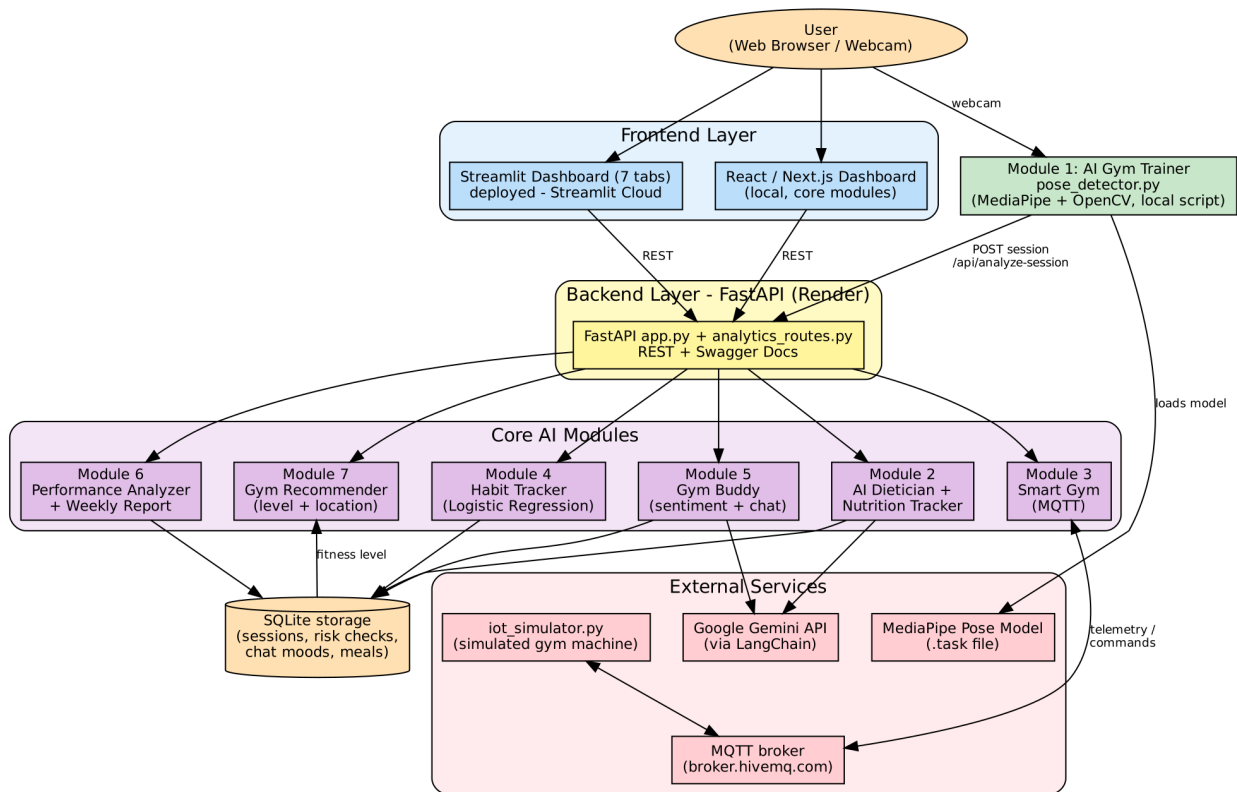


Figure 1: System architecture.

Integration points

- **Module 1 → Module 6:** when a webcam session ends, its reps, duration and form feedback are POSTed to /api/analyze-session, scored and saved.
- **Storage → Modules 6, 7 and Admin Analytics:** saved sessions feed the weekly report, the analytics charts and the recommender's fitness level.
- **Module 3 ↔ MQTT:** the backend reads machine telemetry from the broker and can publish a resistance command back to the machine.
- **Modules 4 and 5 → Storage:** every risk check and chat mood is saved for the analytics view.

4. Technology Stack

Layer	Technology used
Dashboard	Streamlit (deployed), Plotly charts; React / Next.js (local, core modules)
Backend	Python, FastAPI
AI / ML	Google Gemini (via LangChain), MediaPipe Pose Landmarker, OpenCV, scikit-learn (Logistic Regression)
IoT	MQTT (paho-mqtt), public broker broker.hivemq.com, Python machine simulator
Storage	SQLite
Deployment	Render (backend), Streamlit Community Cloud (dashboard), Docker Compose (included)
Version control	Git / GitHub

Note: the original proposal listed a broader stack (TensorFlow/PyTorch, MongoDB/PostgreSQL, Node-RED, Hugging Face/OpenAI, AWS S3/Firebase, D3.js). The final system uses a leaner stack that still meets each module's goal: MediaPipe and scikit-learn for ML, Gemini for generative features, MQTT for IoT, SQLite for storage and Plotly for charts.

5. Core AI Modules

5.1 Module 1 — AI Gym Trainer

A local Python script (`ai_modules/pose_detector.py`) that uses MediaPipe's Pose Landmarker and OpenCV to track the arm through a webcam. It computes the elbow angle, counts bicep curl reps, warns about elbow drift and gives beeps for good reps and for bad form. The rep counter needs several consecutive frames to confirm each movement, a minimum time per rep and good landmark visibility, so small jitter while standing still is not counted. Beeps run in a background thread so the video never freezes. It runs as a script rather than in the browser because real-time webcam processing suits a native program, and it connects to the system through the backend API.

5.2 Module 2 — AI Dietician & Calorie Coach

Calculates BMI and uses Gemini to produce a personalised meal plan and grocery list from weight, height, goal and dietary preference. A nutrition tracker adds a daily calorie target (Mifflin-St Jeor with an activity factor and goal adjustment) and a meal log showing calories eaten and remaining. Tested example: 70 kg, 170 cm, weight loss gives BMI 24.2 (Normal weight) and a target of 1,680 kcal against a maintenance level of 2,180 kcal.

5.3 Module 3 — Smart Gym Assistant (AI + IoT)

A simulated gym machine publishes heart rate, resistance level and status over MQTT. The backend reads these readings and recommends an intensity action, a resistance level and a rest time. An Apply button publishes the new resistance back to the machine, which confirms it received the command. If no machine is publishing, the endpoint falls back to simulated readings and says so (`source=simulated`). Tested live: with the simulator running the dashboard showed `source=mqtt`, and applying resistance 3 was received by the simulator.

5.4 Module 4 — AI Fitness Habit Tracker

A Logistic Regression model (StandardScaler + LogisticRegression, trained on 400 rows of synthetic data) predicts the chance of skipping the next workout from days since the last workout, average sessions per week and average completion. Probabilities map to Low (<0.3), Medium (0.3–0.6) and High (>0.6) risk. Each result comes with a nudge and a suggested schedule. Tested: High (96%) gives a Light 15-minute plan, Medium (58%) gives Moderate 30 minutes, and Low (1%) gives Progressive 45 minutes.

5.5 Module 5 — Virtual Gym Buddy

A Gemini-powered companion with lexicon-based sentiment detection that handles negation ("not happy"), filler words and negative phrases ("don't have mood"). It remembers the last five turns and the analytics view shows the mood trend. When the mood is negative the prompt asks it to reply with empathy and keep the pressure low. Because Gemini writes the reply, the exact advice varies (for example, it may suggest a rest day). Tested: tired and stressed gives negative, great and motivated gives positive, and "I am not happy with my progress" gives negative.

5.6 Module 6 — Pose-to-Performance Analyzer

Scores a session into a Performance Score, Form Quality % and Rating from reps, duration and the share of good-form feedback, and saves it. A weekly report summarises sessions, reps, average score and trend, with charts. Tested: 12 reps in 120 s with 10 of 12 good gives 66.0 / 83.3% / Good; 3 reps in 300 s with 1 of 5 good gives 13.6 / 20.0% / Needs Improvement.

5.7 Module 7 — Gym Recommender & Planner

Recommends programs, a challenge and nearby gyms from the user's goal, streak, fitness level and location. The fitness level comes from saved history: beginner by default, intermediate after 3 or more sessions averaging 45 or more, and advanced after 8 or more sessions averaging 65 or more. Distances to gyms use the haversine formula from a chosen city or coordinates. The gyms are sample data placed around seven real city centres, not live map results.

5.8 Admin Analytics

A dashboard tab that reads the stored data and shows total sessions, risk checks and moods, sessions per week, score trends, risk history and mood trend for a chosen user.

6. Deployment

The backend runs on Render as a FastAPI service with Swagger documentation. The Streamlit dashboard runs on Streamlit Community Cloud and calls the Render backend over public URLs. Both redeploy automatically when changes are pushed to GitHub. The Next.js frontend was run locally and covers the core modules; it does not include the analytics, nutrition tracker or live MQTT features. A Docker Compose configuration is included for containerised use.

Render's free tier sleeps when idle, so the first request after a pause takes about 30–60 seconds. Its disk is also reset on redeploy, so history and meals saved on the live site are not permanent; locally they persist.

7. Testing Summary

Automated: `run_tests.py` sends 23 requests covering every module, including contrasting cases (high / medium / low risk, good / poor session, beginner / advanced streak) and input validation. It passed **23/23 on the local backend and 23/23 on the live Render backend**. The full table is in *testing_report.md*.

Manual: live webcam rep counting with dumbbells (a 26-rep session was counted correctly and standing still gave no phantom reps), Module 1 → 6 integration, all seven dashboard tabs, the MQTT round trip with the simulator, Admin Analytics charts, and the Streamlit Cloud to Render

connection.

Defects found and fixed during testing: video freezing on beeps, phantom reps from landmark jitter, a sentiment miss on "don't have mood", duplicate demo data from concurrent dashboard loads, and a dashboard that showed simulated data because the backend address was not set.

8. Known Limitations

- Nearby gyms are sample data around real city centres, not live map data.
- Render's free tier sleeps when idle and its disk resets on redeploy, so saved history there is not permanent.
- Module 1 runs as a local script, not in the browser.
- The MQTT broker is public; the topic name is unique but not private.
- The Next.js frontend lacks the newest features; the Streamlit dashboard has all of them.
- The skip-risk model is trained on synthetic data, so it demonstrates the method rather than real-world accuracy.
- The performance score weights pace heavily, so slow, controlled sets score low.
- Gym Buddy replies come from Gemini and vary between runs; only the sentiment label is tested automatically.
- Docker files are included, but the live demo uses Render and Streamlit Cloud.

9. Conclusion

The project delivers a single platform that joins workout detection, diet planning, habit prediction, MQTT-connected gym monitoring, conversational motivation and performance analysis. All seven modules are implemented and deployed, usage data is stored and fed back into reports and recommendations, and the system passes 23 of 23 automated tests on both the local and the live backend. It is a working, publicly accessible prototype; the limitations above mark where real data, permanent storage and a fuller web frontend would come next.

10. Repository & Live Links

- Live dashboard (Streamlit): <https://aigymfitnessassistant-sqktqokhufhzdfac6pdefs.streamlit.app>
- Backend API (Render): <https://ai-gym-backend-bwiz.onrender.com>
- API docs (Swagger): <https://ai-gym-backend-bwiz.onrender.com/docs>
- Source repository: https://github.com/Navyachalla21/ai_gym_fitness_assistant